



UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

INGENIERÍA EN DESARROLLO Y GESTION DE SOFTWARE

ASIGNATURA: DESARROLLO WEB PROFESIONAL

8º CUATRIMESTRE GRUPO “A”

ARQUITECTURA DE AUTENTICACIÓN Y CONTROL DE ROLES

PROFR. JOSE MIGUEL CARRERA PACHECHO

LILIA HERNÁNDEZ TUN

Arquitectura de autenticación y control de roles

¿Qué es autenticación y en qué se diferencia de autorización?

La autenticación se encarga de la verificación de la identidad de un usuario y la autorización le proporciona el usuario al nivel adecuado de acceso a los recursos del sistema.

El proceso de autorización se basa en licencias de usuario que describen lo que cada usuario puede hacer dentro de un recurso o red en individual.

¿Cómo funciona la autenticación?

La autenticación, a veces abreviada como "authn", se basa en el intercambio de credenciales de usuario, también llamados factores de autenticación.

Los tipos comunes de factores de autenticación incluyen:

- Factores de conocimiento: algo que solo el usuario sabe, como una contraseña, un PIN o la respuesta a una pregunta de seguridad.
- Factores de posesión: algo que solo tiene el usuario, como un PIN de un solo uso (OTP) enviado a su teléfono móvil personal a través de un mensaje de texto SMS o un token de seguridad física.
- Factores inherentes: biometría, como reconocimiento facial y escaneo de huellas dactilares.

¿Como funciona la autorización?

La autorización, a veces abreviada como "authz", se basa en las licencias de los usuarios. Los permisos son políticas que detallan a qué puede acceder un usuario y qué puede hacer con ese acceso en un sistema.

Los administradores y líderes de seguridad suelen definir las licencias de los usuarios, que luego son aplicados por los sistemas de autorización.

OAuth 2.0, que utiliza tokens de acceso para delegar permisos a los usuarios, es un ejemplo de un protocolo de autorización común. OAuth permite a las aplicaciones compartir datos entre sí.

Tipos de autorización

- Los métodos de control de acceso basado en roles (RBAC) determinan las licencias de acceso de los usuarios en función de sus roles.
- Los métodos de control de acceso basado en atributos (ABAC) emplean los atributos de los usuarios, objetos y acciones, como el nombre de un usuario, el tipo de un recurso y la hora del día, para determinar los niveles de acceso.
- Los sistemas de control de acceso obligatorio (MAC) aplican políticas de control de acceso definidas de forma centralizada a todos los usuarios. Los sistemas MAC son menos granulares que RBAC y ABAC, y el acceso se basa normalmente en niveles de autorización establecido o puntajes de confianza.
- Los sistemas de control de acceso discrecional (DAC) permiten a los propietarios de los recursos establecer sus propias reglas de control de acceso para esos recursos. DAC es más flexible que las políticas generales de MAC.

¿Qué son JWT, sesiones y OAuth?

JWT: Es un estándar para enviar información segura entre partes. No se guarda en el servidor. Esencialmente, JWT se define como un objeto JSON y está protegido con una firma digital, lo que garantiza su integridad y autenticidad.

Sesiones: El servidor crea un registro en su memoria o base de datos cada vez que alguien inicia sesión y le entrega al navegador una "Session ID" guardada en una cookie. En cada petición, el servidor busca esa ID para recordar quién es el usuario.

OAuth: Es un protocolo de autorización que permite a los usuarios de Internet compartir información de forma segura

¿Cuándo conviene usar tokens y cuándo sesiones?

Usa Sesiones si:

- Tu aplicación es monolítica (todo en un mismo servidor).
- Necesitas poder invalidar sesiones de forma inmediata (por ejemplo, "Cerrar sesión en todos los dispositivos").
- Quieres manejar la seguridad principalmente del lado del servidor.

Usa Tokens (JWT) si:

- Tienes una arquitectura de Microservicios o una API que será consumida por una App móvil y una Web.
- Necesitas escalabilidad (no quieres que el servidor guarde millones de sesiones en memoria).
- Estás trabajando con aplicaciones "Single Page" (como React).

¿Qué riesgos de seguridad existen en un mal diseño de autenticación?

Un sistema mal implementado es una puerta abierta para atacantes:

- **Inyección de credenciales:** No limitar los intentos de inicio de sesión (ataques de fuerza bruta).
- **Almacenamiento inseguro:** Guardar contraseñas en texto plano (¡siempre usa hashing como **Bcrypt!**).
- **Exposición de JWT:** Guardar tokens en localStorage los hace vulnerables a ataques XSS (Cross-Site Scripting).
- **No invalidar tokens:** Si un token no expira nunca, un atacante que lo robe tendrá acceso eterno.

¿Cómo se definen roles y permisos en sistemas reales?

En producción se usan modelos:

- **RBAC (Role-Based Access Control):** Se define los roles (Admin, Editor, Usuario) y se les asignan permisos a esos roles.
 - **Users:** Datos básicos (id, nombre, email, password_hash).
 - **Roles:** Los nombres de los niveles (id, nombre como "Administrador", "Editor", "Cliente").
 - **Permissions:** Las acciones atómicas (id, nombre como "create_user", "edit_post", "delete_report").
 - **Role_Permissions:** Una tabla intermedia que vincula qué permisos tiene cada rol.
- **Roles Dinámicos vs. Estáticos**
 - **Estáticos (Hardcoded):** Defines en tu código `if (user.role === 'admin')`. Es fácil de programar, pero poco flexible. Si el cliente quiere un nuevo rol "Supervisor" con permisos mixtos, tienes que reprogramar.
 - **Dinámicos:** El sistema consulta la tabla `Role_Permissions`. Esto permite que, desde un panel de control, el administrador cree nuevos roles y les asigne permisos haciendo "checks" en una lista, sin tocar una sola línea de código.

Definición de Usuarios, Roles y Permisos (RBAC)

Hemos implementado un modelo de **Control de Acceso Basado en Roles (RBAC)** para separar las responsabilidades de cada actor del sistema:

- **Administrador (Admin):**
 - **Permisos:** Gestión total de usuarios (`users:create`, `users:block`), auditoría de bitácoras (`logs:read`) y asignación de roles.
- **Repartidor (Driver):**

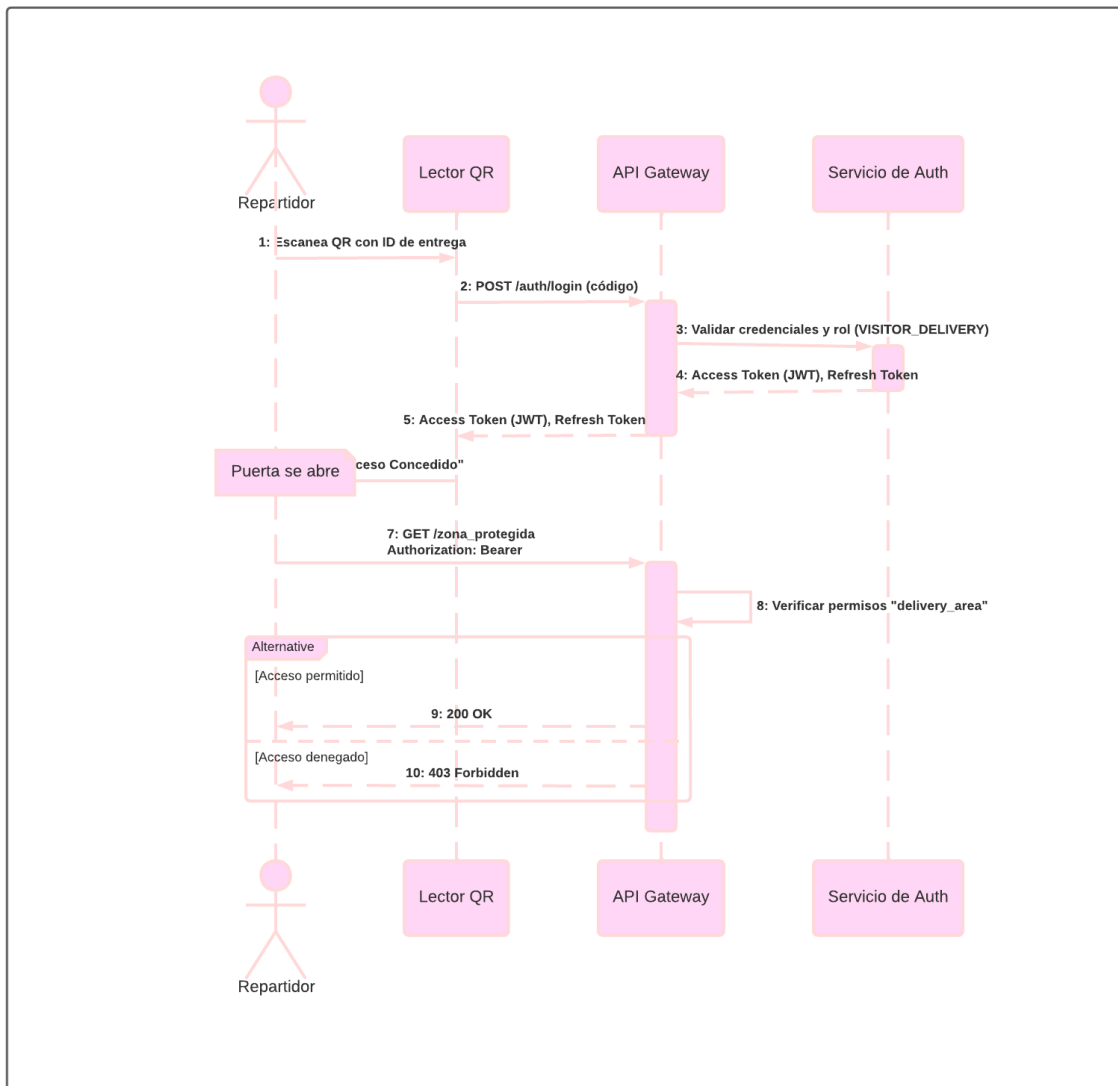
- **Permisos:** Generación de identidad digital (generate_qr), actualización de datos personales y placas, y carga de documentación (INE/Perfil).
- **Guardia de Seguridad (Guard):**
 - **Permisos:** Validación de accesos mediante escaneo (scan_qr), autorización de ingresos (authorize_entry) y consulta de bitácora de turno.
- **Residente:**
 - **Permisos:** Generación de pases temporales para visitas (generate_guest_qr) y consulta de historial propio.

Flujo de Autenticación y Autorización

Para este diseño, se ha seleccionado un enfoque **Stateless** basado en **JWT (JSON Web Tokens)**:

1. **Autenticación (AuthN):** El usuario inicia sesión con sus credenciales (teléfono de 10 dígitos y contraseña). El servidor valida el hash (Bcrypt) y devuelve un JWT que contiene el role_id y user_id.
2. **Generación de Acceso:** El repartidor solicita un pase; el sistema genera un **QR dinámico con expiración de 30 segundos**, mitigando el riesgo de uso de capturas de pantalla.
3. **Autorización (AuthZ):** Al escanear, el **Middleware de Autorización** en el backend verifica que el Guardia tenga los permisos necesarios y que el token del Repartidor sea válido y vigente antes de mostrar la información de acceso.

Sequence Diagram: Authentication & Authorization



Justificación Técnica y Riesgos Mitigados

La decisión de implementar una arquitectura basada en JWT (JSON Web Tokens) y un modelo de Control de Acceso Basado en Roles (RBAC) responde a la necesidad de crear un sistema robusto que cumpla con los estándares actuales de la industria de software:

- **Arquitectura Stateless y Escalabilidad:** Al utilizar JWT, el servidor no necesita mantener una tabla de sesiones activa en memoria. Esto permite

que el backend sea "stateless", facilitando que el sistema escale horizontalmente (añadiendo más servidores) sin perder la trazabilidad de quién está conectado.

- **Seguridad Criptográfica:** Cada token emitido está firmado digitalmente por el servidor. Esto garantiza que, aunque el token viaje por el lado del cliente (React), cualquier intento de modificar el rol o el ID del usuario invalidará la firma, protegiendo la integridad de la sesión.
- **Eficiencia en Micro-validaciones:** El modelo RBAC permite que el sistema tome decisiones de acceso instantáneas consultando los "claims" del token. Esto es vital para que el Guardia pueda validar un ingreso en segundos, sin necesidad de realizar múltiples consultas complejas a la base de datos en cada movimiento.
- **Mitigación de Riesgos Específicos:**
 - **Suplantación de Identidad:** Implementamos un "QR Dinámico" con expiración de 30 segundos. Esto asegura que un código interceptado o una captura de pantalla vieja sea inútil para un acceso no autorizado.
 - **Inyección y Exposición de Datos:** Las contraseñas nunca se almacenan de forma legible; utilizamos el algoritmo Bcrypt para generar hashes seguros con "salting", haciendo que los datos sean inútiles incluso en caso de una filtración de la base de datos.